Simulation-Driven Benchmarking Framework for LiDAR-Based Collision Avoidance on AGVs

Myroslav Mishchuk[1,2], Leonardo Schiavo[3], Milad Jafari Barani[4], S M Asiful Huda[5], and Rafał Cupek[1]
[1]Silesian University of Technology, Poland
[2]Lviv Polytechnic National University, Ukraine
[3]Universidad Politécnica de Madrid, España
[4]University of Oviedo, Spain
[5]University of Naples Federico II, Italy

***Abstract*—LiDAR-based collision avoidance for automated guided vehicles (AGVs) is commonly evaluated using custom scenarios, simulation setups, and metrics. The absence of a standardized evaluation framework makes cross-study comparison and results reproduction difficult. In this paper, we introduce a simulation-driven benchmarking framework for LiDAR-based AGV collision avoidance. The framework combines a parameterized procedural world generator with a modular three-stage simulation engine (detection, fusion, avoidance) and an explicit LiDAR and prior-map degradation model. We validate the framework on a stratified corpus of 480 worlds spanning 48 strata, executing 11,520 simulation runs over eight algorithm pipelines and three sensor configurations. The results indicate that the corpus is well-formed (mean free-space ratio 0.942, 0.925, 0.912 for subdivision levels 1, 2, 3, showing that higher subdivision levels produce more constrained environments; 99.8% of worlds satisfy the clearance bound); the three-stage decomposition exhibits the intended direct-sensitivity pattern and the predicted fusion–avoidance interaction; and pipeline rankings are robust under seed resampling (mean Kendall tau 0.737) and largely stable across sensor/map degradation (mean 0.560), with one metric showing regime-dependent sensitivity that the benchmark localizes explicitly. The framework provides a reproducible, stage-addressable benchmark that isolates each algorithmic choice under controlled perception conditions, enabling cross-study comparison that ad-hoc evaluation practice currently precludes.**

***Index Terms*—Automated Guided Vehicles (AGVs), collision avoidance, benchmarking framework, LiDAR-based navigation**

I. Introduction

Automated Guided Vehicles (AGVs) are widely used in warehouse and manufacturing environments for material transportation, where they increasingly share floor space with human workers [1]. Reliable collision avoidance is the foundational safety mechanism for human–AGV coexistence: failures translate directly into worker-injury risk, and driverless industrial trucks are accordingly governed by dedicated safety standards [2]. AGVs operating in dynamic environments must detect and avoid obstacles such as other vehicles, personnel, and misplaced goods in real time, typically using LiDAR as the primary perception sensor.

A substantial number of papers have been published on collision avoidance for mobile robots with a variety of algorithms proposed at each pipeline stage: obstacle detection via point cloud clustering [3], temporal fusion via tracking filters [4], and local avoidance via reactive planners [5], [6]. More recently, deep reinforcement learning (DRL) methods have been applied to end-to-end collision avoidance [7]. However, a persistent problem in this field is the lack of a standardized evaluation methodology. Each study constructs its own simulation environment, defines its own scenarios, and selects its own metrics, making cross-study comparison unreliable. Adjacent domains have addressed this through shared benchmarks: KITTI [8] and nuScenes [9] for autonomous driving perception, RWARE [10] for multi-robot warehouse coordination, and BARN [11] for metric ground navigation in cluttered spaces. But, to our knowledge, no equivalent benchmark exists for LiDAR-based AGV collision avoidance as a decomposable perception-to-action task with explicit sensor- and map-quality control. Without a shared evaluation framework, it is impossible to determine whether a reported improvement stems from the proposed algorithm or from favorable scenario selection. Researchers cannot reproduce each other's results, and practitioners cannot make informed decisions about which collision avoidance approach to deploy.

In this paper, we propose a simulation-driven benchmarking framework that fills this gap for LiDAR-based AGV collision avoidance. The framework provides (1) a parameterized **world generator `(WG)`** that procedurally produces diverse AGV environments, with control over environment structure, obstacle configurations, and AGV mission parameters; and (2) a modular **simulation engine `(SE)`** with a formally defined perception model and pipeline decomposition, enabling controlled substitution and ablation of methods at each stage. To validate the framework, we address the following research questions (RQs):

- **RQ1:** Does the world generation procedure produce environments that are structurally representative of real AGV operating conditions?
- **RQ2:** Does the pipeline decomposition enable isolated comparison of algorithmic choices at each stage?
- **RQ3:** Are benchmarking results reproducible and stable across seeds, sensor configurations, and map conditions?

The framework is released as open source[1] to support reproduction, extension, and cross-study comparison.

[1]Source code, world corpus, simulation reports, and analysis scripts: https://github.com/Myroslav437/AGV-CABench.

II. Related Works

Industrial mobile robot navigation systems are typically structured as modular pipelines of independent stages. End-to-end approaches that map sensor inputs directly to control actions are an active research direction. However, learned policies pose challenges related to opacity, non-determinism, and verifiability that complicate compliance with safety-critical standards for driverless industrial trucks [2], [12]. Modular pipelines, therefore, remain a popular and practical choice in industrial AGV deployments: frameworks such as ROS2 Nav2 [13] expose each stage as a configurable plugin with well-defined interfaces (for example, costmap layers for perception, pluggable global and local planners for path planning, and controller plugins for motion execution). This lets individual components be tuned, swapped, or validated in isolation, and lets learned methods be introduced at a single stage without replacing the surrounding pipeline.

Within this architecture, LiDAR-based obstacle detection typically relies on point cloud clustering. Euclidean clustering [3] groups points using a fixed distance threshold and is widely used in AGV applications due to its simplicity and low computational cost. DBSCAN [14] and its variants (HDBSCAN, OPTICS) offer greater robustness to noise and varying obstacle densities through neighborhood-based clustering criteria. At the temporal fusion level, Kalman filtering (KF) [4] remains the standard method for tracking detected obstacles across time steps, maintaining position and velocity estimates under noisy observations.

At the reactive planning level, two families dominate. The Vector Field Histogram (VFH) [5] and its successors (VFH+, VFH*) construct polar histograms of obstacle density around the robot and select the best traversable direction. The Dynamic Window Approach (DWA) [6] samples velocity pairs within the robot's kinematic constraints, simulates short-horizon trajectories, and selects the one maximizing a weighted objective of heading, clearance, and speed. Both are extensively deployed in AGV navigation [15], [16] and available in standard robotics middleware. More recently, DRL methods have been applied to collision avoidance [7].

A persistent issue across the collision avoidance literature is the absence of standardized evaluation methodology. Each study evaluates its proposed algorithm in a custom simulation, typically a small set of hand-designed scenarios in Gazebo, Gym, or a custom engine, with idiosyncratic metrics and environmental configurations. This ad-hoc practice is widely reported: Perille et al. [11] observe that navigation systems are routinely compared in one or two manually chosen environments of unquantified difficulty; recent DRL navigation surveys note that evaluations are confined to simplified dynamic scenarios with inconsistent metrics [17]; and researchers at the U.S. National Institute of Standards and Technology (NIST) have explicitly called for standard benchmarking procedures in their assessment of AGV navigation performance [18]. Consequently, when studies report different performance across algorithms, it is difficult to determine whether the differences reflect algorithmic capability or favorable scenario selection. Small scenario counts further limit statistical power, leaving results incomparable across studies.

Adjacent domains have addressed this gap through shared benchmarks. In autonomous driving, KITTI [8] and nuScenes [9] provide standardized datasets, metrics, and leaderboards for perception tasks. In multi-robot coordination, RWARE [10] provides a parameterized warehouse environment for evaluating multi-agent DRL. The closest prior work on ground navigation is BARN [11], which procedurally generates cluttered 2D environments via cellular automaton inspired by search-and-rescue scenarios and scores each as a monolithic sense-plan-act unit with a single learned difficulty value. However, none of these targets LiDAR-based AGV collision avoidance specifically: BARN abstracts away sensing; the autonomous-driving benchmarks address perception in isolation rather than closed-loop avoidance; and RWARE abstracts away both sensing and low-level control.

Our framework instead targets warehouse-style AGV operating conditions and decomposes the pipeline into detection, fusion, and avoidance stages, with explicit LiDAR and prior-map quality parameters. This enables per-stage diagnosis of failures under controlled sensor and map degradation. Crucially, the framework is a formal specification rather than a fixed dataset: world state, perception, and the simulation step are abstract interfaces independent of any particular physics backend. This allows a richer parameter space and on-demand corpus generation at an arbitrary scale.

III. Framework Overview

Addressing the gap identified in Section II requires an evaluation framework that specifically targets LiDAR-based collision avoidance and replaces custom, ad-hoc simulations with standardized scenarios and comparable metrics. We propose such a framework, decoupling world generation, perception-limited simulation, and algorithmic evaluation into independent, composable stages. This separation enables systematic comparison of different algorithm configurations across a large and diverse set of procedurally generated environments. The framework consists of two principal components:

1) **`WG`** — responsible for producing a large corpus of initial world states with varying environment structures, obstacle configurations, and predefined AGV paths, governed by a rich set of generation parameters.
2) **`SE`** — responsible for executing step-wise simulation of the perception-limited AGV navigating through a given world under a specified algorithmic pipeline and sensor configuration, and collecting performance metrics.

Both components are fully parameterized, enabling controlled experimentation over generation conditions, sensor characteristics, and algorithmic configurations simultaneously.

A. Formal Definitions

1) Waypoint Path: describes how an entity is intended to move through the environment: an ordered sequence of positions to visit, with a target speed for each segment. We

define the space of waypoint paths $\mathbb{W}$ as follows. A waypoint path $\mathcal{W} \in \mathbb{W}$ is a tuple:

$$\mathcal{W} = \big(\mathbf{w}_0,\ \mathbf{w}_1,\ \ldots,\ \mathbf{w}_n\ ;\ v_1,\ v_2,\ \ldots,\ v_n\big), \tag{1}$$

where:

- $\mathbf{w}_j \in \mathbb{R}^2$ for $j = 0, 1, \ldots, n$ are the waypoint positions, defining n straight-line segments.
- $v_j \in \mathbb{R}_0^+$ for $j = 1, 2, \ldots, n$ is the target speed for segment j, i.e., the segment from $\mathbf{w}_{j-1}$ to $\mathbf{w}_j$.

We write $|\mathcal{W}| = n$ for the number of segments. The entity following the path traverses the waypoints in order, moving along the segment from $\mathbf{w}_{j-1}$ to $\mathbf{w}_j$ at target speed v_j. The heading along segment j is determined by the segment direction: $\theta_j = \operatorname{atan2}(\mathbf{w}_j - \mathbf{w}_{j-1})$. The trivial case $|\mathcal{W}| = 0$ represents a stationary entity. Higher-level path shapes can be approximated by denser waypoint sequences.

2) Obstacle Model: Each obstacle o_i represents an entity in the environment that the AGV must detect and react to *at runtime*. An obstacle is defined as:

$$o_i = \langle S_i,\ \xi_i \rangle, \tag{2}$$

where:

- $S_i \subset \mathbb{R}^2$ is the spatial extent of the obstacle: a compact, closed region representing the physical space it occupies. Formally, S_i is defined relative to the obstacle's centroid by its boundary:

$$\partial S_i = (\mathbf{b}_1,\ \mathbf{b}_2,\ \ldots,\ \mathbf{b}_{n_i}), \tag{3}$$

  where $\mathbf{b}_k \in \mathbb{R}^2$ are vertices of a closed polygon expressed in the obstacle's local coordinate frame.
- $\xi_i \in \mathbb{W}$ is the trajectory of the obstacle's centroid.

The definition o_i is fixed at the world generation stage. During simulation, the obstacle's state evolves as it progresses along its trajectory. The full obstacle set with time-varying states is:

$$O_t = \{(o_i,\ \tau_i(t))\}_{i=1}^{k}, \tag{4}$$

where $\tau_i(t) \in [0, |\xi_i|]$ is the progress of obstacle i along ξ_i at time t, which determines its current position and heading. For static obstacles ($|\xi_i| = 0$), $\tau_i(t) = 0\ \ \forall\, t$.

3) Environment Structure: The environment structure E represents the static, a priori known physical layout of the operating space, such as walls, columns, shelving units, and other permanent features that define the facility in which the AGV operates. Formally:

$$E = \{e_1,\ e_2,\ \ldots,\ e_l\}, \tag{5}$$

where each e_j is a closed polygonal chain in $\mathbb{R}^2$, representing a contiguous region of occupied space. Elements of E are permanent, known before the AGV begins its mission, and available to the navigation system as part of the prior map.

4) Prior map: The prior map M_0 encodes the AGV's a priori knowledge of the environment, derived from the environment structure E via a degradation function $\mathcal{P}$:

$$M_0 = \mathcal{P}(E,\ \eta_{\text{cpl}},\ \sigma_{\text{map}}), \tag{6}$$

where $\eta_{\text{cpl}} \in [0, 1]$ controls map completeness (the fraction of boundary elements retained) and $\sigma_{\text{map}} \geq 0$ controls positional noise applied to retained elements. With $\eta_{\text{cpl}} = 1$ and $\sigma_{\text{map}} = 0$, M_0 is a copy of E. The degradation parameters are part of the simulation configuration.

The prior map degradation is distinct from LiDAR distortion ($\Lambda_{\mathcal{D}}$). LiDAR noise is local and transient: it affects the current scan and can be filtered over time. Map errors are persistent and structural: a missing wall causes the AGV to plan routes through blocked spaces, and a shifted wall triggers false obstacle detections. These failures affect every decision throughout the mission and cannot be compensated by LiDAR alone, particularly in occluded regions outside the sensor's field of view (FOV). The prior map is computed once at the start of each simulation run and does not change over time.

5) LiDAR Scan: The *ideal scan* at time t is a discretized, range-limited slice of the physical environment as seen from the AGV's current pose:

$$L_t^{\circ} = \mathcal{S}(E,\ O_t,\ P_t,\ \theta_t,\ \Lambda_{\mathcal{S}}) = \{(r_j,\ \alpha_j)\}_{j=1}^{N_{\text{rays}}}, \tag{7}$$

where:

- $P_t \in \mathbb{R}^2$ is the position of the AGV (centroid) at time t.
- $\theta_t \in [0, 2\pi)$ is the heading of the AGV at time t.
- $\Lambda_{\mathcal{S}}$ is LiDAR sensor parameters (see Section III-C1).

For each ray at angle α_j relative to the heading, r_j is the true distance to the nearest surface, whether belonging to E or to an obstacle in O_t. The scan function $\mathcal{S}$ returns perfect, noise-free measurements.

Real LiDAR hardware introduces noise, missed detections, and spurious readings. The distortion function $\mathcal{D}$ transforms the ideal scan into a "realistic" one:

$$\hat{L}_t = \mathcal{D}(L_t^{\circ},\ \Lambda_{\mathcal{D}}) = \{(\hat{r}_j,\ \alpha_j)\}_{j=1}^{N_{\text{rays}}}, \tag{8}$$

where $\Lambda_{\mathcal{D}}$ are the LiDAR distortion parameters (see Section III-C1). The LiDAR observes both obstacles and environment structure indiscriminately.

6) World State: A world state at time t is defined as the tuple:

$$W_t = \langle E,\ O_t,\ \pi,\ P_t,\ S_{\text{agv}},\ \theta_t,\ \mathbf{v}_t \rangle, \tag{9}$$

where:

- $\pi \in \mathbb{W}$ is the time-invariant AGV's reference path that the AGV is expected to follow in the absence of obstacles (1). The first waypoint $\mathbf{w}_0$ of π is the start position; the last waypoint $\mathbf{w}_n$ is the goal.
- $S_{\text{agv}} \subset \mathbb{R}^2$ is the time-invariant spatial extent of the AGV (same as (3)).
- $\mathbf{v}_t \in \mathbb{R}^2$ is the velocity vector of the AGV at time t.

The world state is an "objective" representation of the physical environment and the AGV's mission: it describes what exists and what the AGV is tasked to do, independent of what the AGV perceives or believes at runtime.

7) Perception Model: The AGV does not have direct access to the "objective" world state W_t. Instead, it operates on a perceived world $\tilde{W}_t$:

$$\tilde{W}_t = \Psi(W_t) = \langle M_0,\ \pi,\ S_{\text{agv}},\ \hat{L}_t,\ P_t,\ \theta_t,\ \mathbf{v}_t \rangle, \tag{10}$$

where Ψ is a perception function that transforms the objective world through the sensors and prior knowledge. We assume the AGV has perfect knowledge of its own pose: P_t, θ_t, $\mathbf{v}_t$ pass through Ψ unchanged. Relaxing this assumption is left to future work (Section V-D).

B. World Generator (WG)

The WG constructs initial world states W_0. A single world is generated as:

$$\texttt{WG}(\Omega,\ s_{\text{wg}}) \to W_0, \tag{11}$$

where WG is presented as stochastic generation function parameterized by a parameter set Ω and seeded by $s_{\text{wg}} \in \mathbb{N}$, which initializes the pseudo-random number generator:

$$\Omega = \langle \Omega_{\text{env}},\ \Omega_{\text{agv}},\ \Omega_{\text{path}},\ \Omega_{\text{obs}} \rangle. \tag{12}$$

The generation proceeds in five stages: (1) the environment structure E is generated via *Binary Space Partitioning* (BSP) from Ω_{env}; (2) the AGV spatial extent S_{agv} is constructed from Ω_{agv}; (3) the AGV reference path π is generated via *Probabilistic Roadmap* (PRM) within the free space of E according to Ω_{path}; (4) static and dynamic obstacles are placed in the free space of E according to Ω_{obs}, respecting clearance constraints relative to boundary geometry and the reference path; (5) trajectories for dynamic obstacles are generated via PRM within the free space of E. The corpus of N initial worlds is produced using WG:

$$\mathcal{C} = \{W_0^{(i)}\}_{i=1}^{N}, \quad N = 480. \tag{13}$$

To ensure coverage over the parameter space, the corpus is stratified: Ω is systematically varied across predefined ranges (see Section IV), so that each region of the parameter space is represented by a sufficient number of worlds.

1) Environment parameters Ω_{env}: These parameters control the generation of the environment structure E. The generation of boundary geometry depends on the chosen spatial partitioning algorithm; in this work, we employ BSP, a well-established recursive subdivision method widely used in procedural environment generation [19]. The BSP algorithm recursively divides the environment into axis-aligned rectangular partitions; boundary elements (walls) are placed along partition edges, and rooms or passages are carved within leaf partitions. By varying the BSP parameters, a single algorithm produces structurally diverse layouts, from open spaces to dense room-based facilities. The environment parameters are:

- $X, Y \in \mathbb{R}^+$ — spatial dimensions of the environment.
- $d_{\text{bsp}} \in \mathbb{N}^+$ — the number of recursive subdivision levels. Low values (1–2) yield open layouts with few large partitions; medium values (3–4) produce corridor- or aisle-like structures; high values (5+) generate dense, room-based environments.
- $s^w_{\min},\ s^h_{\min} \in \mathbb{R}^+$ — the smallest allowable width and height for a leaf partition. These prevent degenerate (too narrow or too small) regions and indirectly control room size.
- $\rho_{\text{split}} \in [\rho_{\min},\ \rho_{\max}] \subset (0, 1)$ — split ratio range. Narrow ranges (e.g., $[0.45, 0.55]$) produce regular, aisle-like partitions; wide ranges (e.g., $[0.2, 0.8]$) produce heterogeneous room sizes.
- $p_{\text{room}} \in [p_{\min},\ p_{\max}] \subset [0, 1)$ — room padding range. Low padding yields rooms that nearly fill their partitions (dense layout); high padding yields smaller rooms with more open space.
- $c_{\min} \in \mathbb{R}^+$ — minimum corridor width between adjacent boundary elements (room walls and partition walls).
- $w_{\text{pass}} \in \mathbb{R}^+$ — the width of corridors connecting sibling partitions. Controls navigability and the effective aisle width.
- $t_{\text{wall}} \in \mathbb{R}^+$ — the thickness of boundary elements placed along partition edges and room perimeters.

The BSP algorithm outputs a set of closed polygonal chains (the walls and room boundaries), which constitutes the environment structure E. Different parameter configurations are used to stratify the world corpus across layout types (see Section IV).

2) AGV parameters Ω_{agv}: These parameters define the physical properties of the AGV: the body length $L_{\text{agv}} \in \mathbb{R}^+$ and body width $W_{\text{agv}} \in \mathbb{R}^+$.

3) Path parameters Ω_{path}: These parameters control the generation of the reference path $\pi \in \mathbb{W}$. As with environment generation, path generation depends on the chosen algorithm; in this work, we employ the PRM method, a well-established sampling-based path planning algorithm widely used in robotics [20]. The path generation proceeds as follows: (1) inflate the boundary elements of E by a clearance margin d_{clear}; (2) sample n_{samples} random points in the inflated free space; (3) connect each sample to its k_{nn} nearest neighbors via collision-free straight-line edges; (4) sample start and goal positions in the free space; (5) insert start and goal into the roadmap and find the shortest path via Dijkstra's algorithm; (6) assign target speeds to each segment from $\mathcal{D}_{v^\star}$. The path parameters are:

- $n_{\text{samples}} \in \mathbb{N}^+$ — number of random points sampled in the free space (controls roadmap density and path diversity).
- $k_{\text{nn}} \in \mathbb{N}^+$ — number of nearest neighbors to attempt connections with (controls roadmap connectivity).
- $l_{\max} \in \mathbb{R}^+$ — maximum edge length; connections longer than $l_{\max}$ are rejected.
- $d_{\text{clear}} \in \mathbb{R}^+$ — wall clearance distance (obstacle inflation radius); the generated path is guaranteed to maintain at least this distance from any boundary element in E.
- $l^{\min}_{\pi} \in \mathbb{R}^+$ — minimum path length.
- $\mathcal{D}_{v^\star}$ — distribution over target segment speeds (e.g., $\text{Uniform}(v^\star_{\min}, v^\star_{\max})$, or constant speed across all segments).

4) Obstacle parameters Ω_{obs}: These parameters control the generation of the obstacle set $O_{t=0}$. In the current implemen-

tation, obstacle shapes are restricted to circles and squares, both parameterized by a single scalar: the inscribed circle radius r_i. Trajectories for dynamic obstacles are generated using the same PRM-based approach as the AGV reference path, with their own parameter set. The complete parameter set for obstacles is:

- $n_{\text{static}} \in \mathbb{N}$ — number of static obstacles.
- $n_{\text{dynamic}} \in \mathbb{N}$ — number of dynamic obstacles.
- $\mathcal{D}_r$ — distribution over inscribed circle radius (e.g., $\text{Uniform}(r_{\min}, r_{\max})$). For circles, r_i is the radius directly; for squares, r_i is the half-side length (the radius of the inscribed circle).
- $d_{\min}$ — minimum clearance between any two obstacle boundaries and between obstacles and environment boundary geometry.
- $n^{\text{obs}}_{\text{samples}}$, $k^{\text{obs}}_{\text{nn}}$, $l^{\text{obs}}_{\max}$, $d^{\text{obs}}_{\text{clear}}$ — PRM roadmap parameters for obstacle trajectory generation (analogous to the AGV path parameters; may differ to produce shorter or less constrained paths).
- $\mathcal{D}^{\text{obs}}_v$ — distribution over segment target speeds for dynamic obstacles (e.g., $\text{Uniform}(v_{\min}, v_{\max})$).

For static obstacles, the trajectory is simply a single waypoint at the obstacle's initial position: $\xi_i = (\mathbf{w}_0)$ with $|\xi_i| = 0$.

C. Simulation Engine (SE)

Each simulation run is fully specified by:

$$\texttt{SE}(W_0,\ \mathcal{A},\ \Sigma,\ s_{\text{se}}) \to \mathcal{M}, \tag{14}$$

where $W_0 \in \mathcal{C}$ is an initial world from the generated corpus, $\mathcal{A}$ is the pipeline configuration, Σ is the simulation parameter set, $s_{\text{se}} \in \mathbb{N}$ is the seed for the pseudo-random number generator, and $\mathcal{M}$ is the resulting set of metrics. The core simulation loop is governed by:

$$W_t = f(W_{t-1}\ ;\ \mathcal{A},\ \Sigma)\,. \tag{15}$$

At each step, the SE internally:

1) applies the perception function $\Psi(W_{t-1})$ to produce the perceived world $\tilde{W}_{t-1}$;
2) feeds $\tilde{W}_{t-1}$ to the pipeline $\mathcal{A}$, which produces a control action;
3) advances the world dynamics: the AGV moves according to control action, and all obstacles progress along their trajectories, yielding W_t.

The simulation function f is thus a closed-loop mapping from objective world state to objective world state. The perception function Ψ and the pipeline $\mathcal{A}$ operate inside f; the AGV never has access to W_{t-1} directly, only to $\tilde{W}_{t-1}$. Figure 1 summarizes one application of (15): the perception function Ψ consumes the objective world W_{t-1}, the three-stage pipeline $\mathcal{A}$ operates on the perceived world $\tilde{W}_{t-1}$, and the resulting control action together with the dynamics parameters advances the simulation to W_t. The figure also marks the points at which each component of Σ enters the loop, which the following subsections define.

The SE is governed by a parameter set Σ:

$$\Sigma = \langle \Lambda,\ \Phi,\ \Delta \rangle. \tag{16}$$

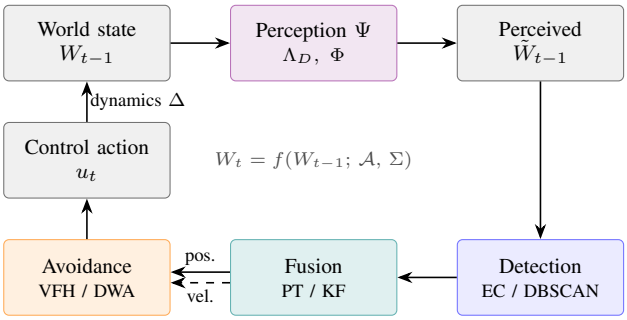


Fig. 1. Simulation engine: one step of the core simulation loop (15).

1) LiDAR sensor parameters Λ: The LiDAR parameters are partitioned into two subsets $\Lambda = \langle \Lambda_{\mathcal{S}},\ \Lambda_{\mathcal{D}} \rangle$:

The *geometric parameters* $\Lambda_{\mathcal{S}}$:

- $r_{\min}, r_{\max} \in \mathbb{R}^+$ — minimum and maximum detection ranges.
- $\theta_{\text{fov}} \in (0, 2\pi]$ — angular FOV.
- $\delta_\theta \in \mathbb{R}^+$ — angular resolution.
- $f_{\text{scan}} \in \mathbb{R}^+$ — scan frequency (Hz).

The *distortion parameters* $\Lambda_{\mathcal{D}}$:

- $\sigma_r \geq 0$ — additive Gaussian range noise: $\hat{r}_j = r_j + \epsilon_j$, with $\epsilon_j \sim \mathcal{N}(0, \sigma_r^2)$.
- $p_{\text{miss}} \in [0, 1)$ — probability that a ray returns no reading despite a surface being in range.
- $p_{\text{ghost}} \in [0, 1)$ — probability of a spurious reading with no real surface.

2) Perception and fusion parameters Φ:

- $\eta_{\text{cpl}} \in [0, 1]$ — map completeness. The fraction of boundary elements in E that are represented in the AGV's prior map. A value of $\eta_{\text{cpl}} = 1$ yields a perfect map; lower values simulate stale or incomplete maps.
- $\sigma_{\text{map}} \geq 0$ — map noise. The positional perturbation applied to boundary elements in the prior map, modeling mapping inaccuracies.

The prior map M_0 is derived from E as part of the perception function using the degradation parameters η_{cpl} and σ_{map} above. During simulation, the SE additionally maintains a runtime map M_t that evolves as the fusion stage integrates new sensor observations with M_0. The runtime map is engine-internal state: it is not part of either W_t or $\tilde{W}_t$, but is available to the fusion stage at each step.

3) Simulation dynamics parameters Δ: include $v^{\max}_{\text{agv}} \in \mathbb{R}^+$ — maximum AGV velocity; $a^{\max}_{\text{agv}} \in \mathbb{R}^+$ — maximum AGV acceleration; $\omega^{\max}_{\text{agv}} \in \mathbb{R}^+$ — maximum AGV angular velocity; and $T_{\text{horizon}} \in \mathbb{R}^+$ — maximum simulation duration.

IV. Experimental Design

For every combination of initial world $W_0^{(i)} \in \mathcal{C}$, pipeline $\mathcal{A}^{(j)} \in \mathbb{A}$, and simulation configuration $\Sigma^{(k)} \in \mathbb{S}$, a simulation run $\texttt{SE}(W_0^{(i)}, \mathcal{A}^{(j)}, \Sigma^{(k)}, s_{\text{se}}^{(i,j,k)}) \to \mathcal{M}^{(i,j,k)}$ is executed and metrics are recorded. The protocol proceeds as: (1) generate the world corpus $\mathcal{C}$; (2) run all $(W_0, \mathcal{A}, \Sigma)$ triples; (3) aggregate metrics; (4) perform the validation analyses.

TABLE I
WORLD GENERATION PARAMETERS

Parameter	Values
Environment (Ω_{env})	
Dimensions $X \times Y$	30×30 m
BSP depth d_{bsp}	1, 2, 3
Min partition $s^w_{\min} \times s^h_{\min}$	6×6 m
Split ratio ρ_{split}	$[0.3, 0.7]$
Room padding p_{room}	$[0.15, 0.35]$
Min corridor $c_{\min}$	2.0 m
Passage width w_{pass}	4.0 m
Wall thickness t_{wall}	0.3 m
AGV (Ω_{agv})	
Body $L_{\text{agv}} \times W_{\text{agv}}$	0.6×0.4 m
Path (Ω_{path})	
PRM samples n_{samples}	500
Neighbors k_{nn}	10
Max edge $l_{\max}$	5.0 m
Wall clearance d_{clear}	0.5 m
Min path length $l^{\min}_{\pi}$	15.0 m
Target speed $v^{\star}$	1.0 m/s (const.)
Obstacles (Ω_{obs})	
Static count n_{static}	0, 4, 8, 16
Dynamic count n_{dynamic}	0, 2, 4, 8
Inscribed radius $\mathcal{D}_r$	$\text{Uniform}(0.1, 0.4)$ m
Min clearance $d_{\min}$	0.3 m
Obs. PRM samples	300
Obs. speed $\mathcal{D}^{\text{obs}}_v$	$\text{Uniform}(0.2, 0.8)$ m/s

A. Pipeline Configurations

To validate the decomposition, we instantiate the three-stage pipeline $\mathcal{A} = \langle A_{\text{det}}, A_{\text{fus}}, A_{\text{avoid}} \rangle$ with two algorithms per stage, yielding $2 \times 2 \times 2 = 8$ configurations. All algorithms below are provided as built-in plugins in the released framework. Users may register additional algorithms via an abstract stage interface.

1) Detection (A_{det}): The detection stage receives the distorted scan $\hat{L}_t$ and the prior map M_0, filters out points matching known boundary geometry, and groups remaining points into obstacle candidates:

- **Euclidean Clustering (EC)**: groups points using a fixed distance threshold d_{ec} [3]. Lightweight and widely used in AGV obstacle detection.
- **DBSCAN**: density-based clustering with neighborhood radius ε and minimum cluster size. More robust to noise and varying obstacle densities [14].

2) Fusion (A_{fus}): The fusion stage integrates current detections with historical observations:

- **Pass-through (PT)**: no temporal fusion; $\bar{O}_t = \hat{O}_t$ directly. Minimal computation baseline. Because it does not estimate velocity, its output's velocity field is set to zero; downstream stages that consume this field hence operate under a stationary-obstacle assumption.
- **Kalman Filter (KF)**: maintains position and velocity estimates per tracked obstacle using a constant-velocity model and nearest-neighbor association [4].

3) Avoidance (A_{avoid}): The avoidance stage computes a control action given the fused obstacle state and reference path:

- **Vector Field Histogram (VFH)**: constructs a polar histogram of obstacle density and selects the best traversable direction [5]. It is worth noting that VFH is insensitive to velocity estimates from the fusion stage, as it considers only the obstacle's estimated position and radius.
- **Dynamic Window Approach (DWA)**: samples velocity pairs within kinematic constraints and selects the trajectory maximizing a weighted objective of heading, clearance, and speed [6].

These configurations directly target RQ2: the permutation analysis (Section IV-E) holds two stages fixed and varies the algorithm at the target stage, isolating each stage's contribution to end-to-end performance.

B. World Corpus

The world corpus $\mathcal{C}$ is generated by stratifying Ω across the values in Table I. The full factorial over BSP depth (3), static obstacle count (4), and dynamic obstacle count (4) yields $3 \times 4 \times 4 = 48$ strata. For each stratum, multiple worlds are generated with independent random seeds.

C. Simulation Parameter Configurations

To assess result stability, we define a nominal simulation configuration and two degraded variants (Table II). The `Nominal` configuration represents a well-calibrated LiDAR with a perfect prior map: the "ideal" operating condition. `Degraded-1` introduces moderate sensor noise, reduced range, and minor map incompleteness. `Degraded-2` represents a challenging scenario with significant noise, narrow FOV, coarse resolution, and a substantially incomplete map. The full Ω and Σ parameter sets are exposed through YAML configuration files with documented defaults.

TABLE II
SIMULATION PARAMETER CONFIGURATIONS

Parameter	`Nominal`	`Degraded-1`	`Degraded-2`
LiDAR geometric ($\Lambda_{\mathcal{S}}$)			
$r_{\max}$	10 m	5 m	5 m
$r_{\min}$	0.05 m	0.05 m	0.05 m
θ_{fov}	270°	270°	180°
δ_θ	0.5°	1°	2°
f_{scan}	15 Hz	15 Hz	10 Hz
LiDAR distortion ($\Lambda_{\mathcal{D}}$)			
σ_r	0	0.02 m	0.05 m
p_{miss}	0	0.02	0.05
p_{ghost}	0	0.01	0.02
Perception (Φ)			
η_{cpl}	1.0	0.9	0.7
σ_{map}	0	0.05 m	0.1 m
Dynamics (Δ)			
$v^{\max}_{\text{agv}}$	1.5 m/s	1.5 m/s	1.5 m/s
$a^{\max}_{\text{agv}}$	1.0 m/s^2	1.0 m/s^2	1.0 m/s^2
$\omega^{\max}_{\text{agv}}$	1.0 rad/s	1.0 rad/s	1.0 rad/s
T_{horizon}	120 s	120 s	120 s

D. Evaluation Metrics

For each simulation run, the following metrics are collected:

- μ_{col} — collision rate: fraction of runs in which the AGV collides with an obstacle or boundary.
- μ_{dev} — path deviation: mean distance from the AGV's trajectory to the nearest segment of π.

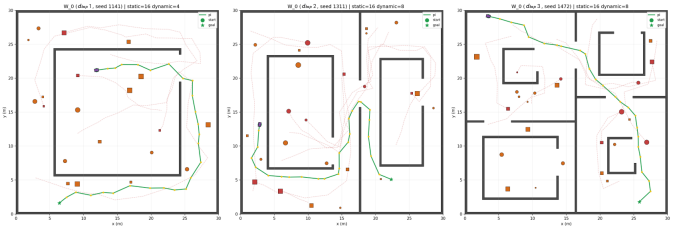


Fig. 2. Example worlds generated at $d_{\text{bsp}} \in \{1, 2, 3\}$ (left to right): environment boundaries E (black), AGV reference path π (green) with start (•) and goal (⋆), static obstacles (orange), dynamic obstacles (red) and dynamic-obstacle trajectories (red, dashed).

- μ_{goal} — goal reach rate: fraction of runs in which the AGV reaches $\mathbf{w}_n$ within T_{horizon}.
- μ_{vel} — dynamic-obstacle velocity RMSE: root-mean-square error, averaged over dynamic obstacles and simulation steps, between the fusion-stage velocity estimate and the ground-truth velocity obtained from the obstacle's trajectory ξ_i.
- μ_{comp} — computational cost: mean CPU time per simulation step.
- μ_{lat} — latency: 95th-percentile per-step decision time.

E. Validation Analyses

To address the outlined RQs, we perform the following analyses on the collected metrics:

1) RQ1 — Environment Realism: For each generated world we compute the free-space ratio (fraction of $X \times Y$ not occupied by boundary geometry) and the minimum passage width (the narrowest gap between any two boundary segments on the AGV's reference path π), and perform:

- *Parameter-consistency check:* free-space ratio should decrease monotonically with d_{bsp}, and the minimum passage width should cluster near $w_{\text{pass}} - t_{\text{wall}}$ by construction.
- *Navigability-safety check:* the minimum passage width should exceed the AGV footprint plus clearance, $W_{\text{agv}} + 2\,d_{\text{clear}}$, in every retained world.
- *Difficulty-spread:* during the runs the μ_{col} and μ_{dev} should rise and μ_{goal} should fall with increasing obstacle count.

2) RQ2 — Decomposition Validity: We run algorithm permutations over the three stages. A valid decomposition requires two properties: (i) *direct stage sensitivity*, i.e., varying each algorithm primarily shifts its sensitivity-matched metric (detection → μ_{col}, fusion → μ_{vel}, avoidance → μ_{dev}, μ_{goal}); (ii) *predictable stage interactions*, i.e., where an avoidance algorithm consumes a field produced by fusion (DWA's forward-prediction consumes the velocity field; VFH does not), the fusion-switch effect on avoidance-sensitive metrics should concentrate in the DWA-based pipelines.

3) RQ3 — Result Stability: We compute the ranking of the 8 pipeline configurations under varying conditions, and perform:

- *Seed-stability check:* under the `Nominal` configuration, rankings should agree across disjoint subsets of $\mathcal{C}$, confirming that the benchmark's verdict does not depend on the particular draw of world seeds.
- *Sensor-and-map-axis stability check:* per-metric rankings should persist across $\Sigma^{(k)}$; breakdowns on a particular metric localize the regime in which that metric's ranking is fragile.

V. Results and Discussion

We instantiate the corpus $\mathcal{C}$ with $N = 480$ worlds (10 per stratum over the 48 strata of Table I) and execute every $(W_0, \mathcal{A}, \Sigma)$ triple, producing $480 \times 8 \times 3 = 11{,}520$ runs. Figure 2 shows three representative outputs of the `WG`. As can be observed, the same generator produces layouts from open halls to room-based facilities. Table III summarizes the aggregate metrics; subsequent subsections analyze them against each research question.

A. Environment Realism

Figure 3 shows the distributions of free-space ratio and minimum passage width across $\mathcal{C}$, grouped by BSP depth. The mean free-space ratio is 0.942, 0.925, and 0.912 for

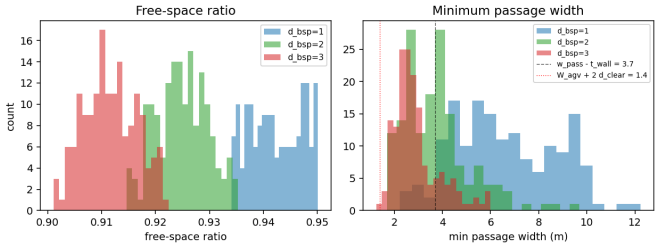


Fig. 3. Distributions of free-space ratio (left) and minimum passage width (right) across $\mathcal{C}$, stratified by $d_{\text{bsp}} \in \{1, 2, 3\}$.

$d_{\text{bsp}} = 1, 2, 3$ respectively, confirming the expected monotonic decrease. The minimum passage width concentrates at ≈ 4.41 m, near the theoretical $w_{\text{pass}} - t_{\text{wall}} = 3.7$ m. With $W_{\text{agv}} + 2\,d_{\text{clear}} = 1.4$ m, 99.8 % of generated worlds satisfy the clearance bound; worlds violating it are retained and inspected. Figure 4 shows the travel metrics as functions of n_{static} and n_{dynamic}, averaged over all `Nominal` runs.

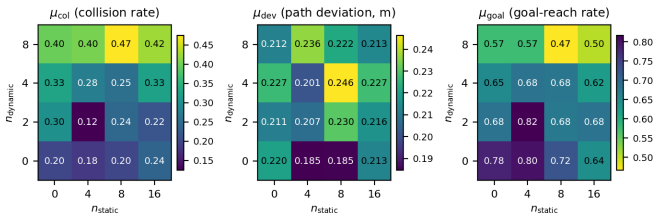


Fig. 4. Travel metrics as a function of obstacle counts under `Nominal`. Each cell aggregates the mean over all 8 pipelines and 10 world seeds in the $(n_{\text{static}}, n_{\text{dynamic}})$ stratum.

Spearman correlations identify n_{dynamic} as the dominant difficulty axis ($\rho = +0.182$ with μ_{col} and $\rho = -0.160$ with μ_{goal}, both $p < 10^{-3}$), while n_{static} is comparatively flat at this scale

TABLE III
AGGREGATE METRICS ACROSS THE 8 PIPELINES, WITH EACH METRIC REPORTED UNDER THE THREE SIMULATION CONFIGURATIONS (MEANS OVER $\mathcal{C}$)

	μ_{col}			μ_{dev}			μ_{goal}			μ_{vel}			μ_{comp}			μ_{lat}		
Pipeline	N	D1	D2	N	D1	D2	N	D1	D2	N	D1	D2	N	D1	D2	N	D1	D2
EC + PT + VFH	0.144	0.204	0.200	0.090	0.200	0.144	0.858	0.696	0.721	0.463	0.451	0.451	0.051	0.026	0.011	0.011	0.005	0.003
EC + PT + DWA	0.450	0.404	0.342	0.332	0.290	0.279	0.438	0.352	0.329	0.454	0.434	0.423	0.074	0.040	0.026	0.056	0.038	0.042
EC + KF + VFH	0.146	0.208	0.190	0.090	0.173	0.149	0.856	0.717	0.723	0.437	0.456	0.527	0.049	0.024	0.010	0.010	0.004	0.003
EC + KF + DWA	0.410	0.344	0.310	0.351	0.304	0.300	0.487	0.383	0.356	0.429	0.428	0.460	0.078	0.041	0.028	0.062	0.041	0.042
DB + PT + VFH	0.146	0.198	0.217	0.091	0.200	0.153	0.856	0.700	0.698	0.463	0.452	0.448	0.061	0.053	0.033	0.049	0.059	0.049
DB + PT + DWA	0.448	0.375	0.327	0.332	0.300	0.268	0.440	0.377	0.304	0.453	0.433	0.408	0.089	0.073	0.051	0.096	0.089	0.082
DB + KF + VFH	0.140	0.200	0.219	0.091	0.203	0.158	0.863	0.702	0.696	0.436	0.451	0.554	0.062	0.054	0.036	0.051	0.061	0.054
DB + KF + DWA	0.410	0.338	0.323	0.349	0.311	0.300	0.485	0.425	0.331	0.429	0.424	0.479	0.087	0.070	0.050	0.092	0.086	0.080

DB = DBSCAN, N = `Nominal`, D1 = `Degraded-1`, D2 = `Degraded-2`.

with only μ_{goal} showing a small but significant $\rho = -0.061$ ($p < 10^{-3}$); μ_{col} still spans 0.200–0.425 across the obstacle-count range. Together, these checks support the interpretation that the corpus is well-formed, physically navigable, and spans a meaningful difficulty range.

B. Decomposition Validity

Table IV reports the per-world median paired difference induced by switching each stage's algorithm, with the other two stages held fixed, under the `Nominal` configuration. Paired-difference significance is assessed with the two-sided Wilcoxon signed-rank test: for each table row the reported p is the probability that a paired shift at least as large as the bolded Δ would arise under the null hypothesis that the algorithm switch has no effect on the bolded metric.

TABLE IV
PER-STAGE EFFECTS UNDER ALGORITHM PERMUTATION (NOMINAL)

Stage (switch)	$\Delta\mu_{\text{col}}$	$\Delta\mu_{\text{vel}}$	$\Delta\mu_{\text{dev}}$	$\Delta\mu_{\text{goal}}$	p
Detection (EC→DB)	**-0.002**	-0.000	-0.000	0.001	0.439
Fusion (PT→KF)	-0.020	**-0.026**	0.009	0.025	<0.001
| Avoidance = VFH	-0.002	-0.027	0.000	**0.002**	0.715
| Avoidance = DWA	-0.039	-0.025	0.018	**0.048**	<0.001
Avoidance (VFH→DWA)	0.286	-0.009	**0.251**	**-0.396**	<0.001

Median paired difference across $\mathcal{C}$. Bold entries denote the metric each row is expected to move: the stage's *sensitivity-matched* metric for the three main rows, and the *interaction-target* metric μ_{goal} for the two avoidance-sliced rows.

Property (i) partially holds: fusion and avoidance shift their sensitivity-matched metrics at $p < 10^{-3}$, whereas detection's μ_{col} shift of -0.002 is below the statistical resolution of this corpus ($p = 0.439$). Property (ii) holds: the fusion-switch effect on μ_{goal} is 0.002 ($p = 0.715$) under VFH and 0.048 ($p < 10^{-3}$) under DWA, consistent with DWA's consumption of the velocity field and VFH's non-consumption.

C. Result Stability

Table V reports Kendall's rank-correlation coefficient τ between pairs of per-metric pipeline rankings. The *Seed (shards)* column is the mean pairwise τ over 3 disjoint `Nominal` seed shards (probing seed dependence); the remaining columns pairwise-compare rankings under `Nominal`, `Degraded-1`, and `Degraded-2` (probing sensor/map-degradation dependence).

Rankings are metric-dependent: the minimum pairwise $\tau = -0.286$ occurs for μ_{vel} on Nom–Deg2, while every other travel metric retains $\tau \geq 0.618$ on the same pair. A plausible mechanism is DWA's run-timeout rate, which rises from 10.8% under `Nominal` to 34.4% under `Degraded-2` (VFH stays below 10%): DWA pipelines then accumulate fewer dynamic-obstacle tracking steps, re-weighting μ_{vel}'s step-averaged RMSE in a way that does not track fusion quality. We interpret this as a regime-dependent sensitivity of μ_{vel}, surfaced by the benchmark rather than a failure of it.

TABLE V
PAIRWISE KENDALL'S τ BETWEEN PIPELINE RANKINGS PER METRIC

Metric	**Seed (shards)**	**Nom–Deg1**	**Nom–Deg2**	**Deg1–Deg2**
μ_{col}	0.715	0.815	0.667	0.571
μ_{dev}	0.667	0.857	0.857	0.714
μ_{goal}	0.707	0.691	0.618	0.643
μ_{vel}	0.857	0.500	-0.286	0.071

Values near $+1$ denote rank agreement; values near -1 denote reversal. The Seed (shards) column is the mean over the 3 disjoint `Nominal` shard pairings; the remaining columns compare rankings between `Nominal`, `Degraded-1`, and `Degraded-2`.

Pooling the Seed (shards) column yields an overall mean $\tau = 0.737$, indicating that the ranking is robust to the particular draw of world seeds.

D. Discussion

The three analyses jointly support the framework's use as a benchmarking instrument: the corpus is well-formed and difficulty-graded (RQ1), the pipeline stages exhibit the intended sensitivity pattern and surface predicted stage interactions (RQ2), and rankings are stable enough for the benchmark's verdict to be reproducible (RQ3). Under `Nominal`, Table III additionally orders pipelines by cost: VFH runs at $\mu_{\text{comp}} \approx 0.049$–$0.062$ s and $\mu_{\text{lat}} \approx 0.010$–$0.051$ s versus DWA's 0.074–0.089 s and 0.056–0.096 s, and EC detection is consistently cheaper than DBSCAN at fixed fusion and avoidance (e.g., $\mu_{\text{comp}} = 0.049$ vs. 0.062 s for KF+VFH). DWA's higher collision and timeout rates in this corpus reflect the algorithm's known sensitivity to objective-weight tuning rather than a general capability gap; surfacing such tuning

sensitivity per stage and per regime is itself one of the framework's intended outputs. The 11,520-run sweep ran in $\sim$8 wall-clock hours on a 16-core workstation, scaling linearly with available cores since runs are independent. Where checks are only partially met, the framework's parameter-controlled design localizes the cause to a specific stage, metric, or regime, rather than leaving it as an uncharacterized source of variance — which is the property that distinguishes a benchmarking instrument from a single-scenario evaluation.

Several limitations shape the scope of these results. First, as noted in Section III-A7, the AGV's self-pose is treated as oracle-known inside $\tilde{W}_t$; real localization uncertainty (odometry drift, SLAM error) is not modeled here. Second, the RQ1 checks establish internal consistency and navigability rather than statistical equivalence to a specific empirical warehouse population, which would require curated measurements from real facilities. Sim-to-real transfer on a physical AGV testbed thus remains the principal question of external validity. Third, the framework currently models a single AGV, with multi-AGV coordination being a natural extension. A pose-noise model, processing-latency modeling, large-scale algorithm benchmarking, per-stratum conditional analysis, sim-to-real transfer, and comparison with external simulators are planned for the journal extension.

VI. Conclusion

In this work, we present a simulation-driven benchmarking framework for LiDAR-based collision avoidance on AGVs, comprising a parameterized procedural world generator and a modular three-stage simulation engine with an explicit sensor- and map-degradation model. Validation on a stratified corpus showed that the generated environments are well-formed and cover a meaningful difficulty range; that each pipeline stage shifts its intended metric in isolation, with detection below the corpus's statistical resolution; and that pipeline rankings are robust to world-seed resampling and largely stable across sensor and map degradation. Unlike a fixed dataset, the framework is a formal specification: it supports on-demand corpus regeneration, controlled stage substitution, and clean separation of perception-side from algorithmic failures. Planned extensions include AGV self-pose uncertainty, processing-latency modeling, per-stratum conditional analysis, integration of learned stages, and cross-simulator validation against external physics backends.

Acknowledgment

Co-funded by the European Union HORIZON TMA MSCA Doctoral Networks / HORIZON-MSCA-2023-DN-01 / project TUAI / grant agreement N° 101168344. Views and opinions expressed are, however, those of the author(s) only and do not necessarily reflect those of the European Union or European Research Executive Agency. Neither the European Union nor the granting authority can be held responsible for them.

During the preparation of this work, the authors used *Anthropic Claude Opus 4.7* for proofreading, text readability improvement, and partial code generation. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

[1] G. Fragapane, R. de Koster, F. Sgarbossa, and J. O. Strandhagen, "Planning and control of autonomous mobile robots for intralogistics: Literature review and research agenda," *European Journal of Operational Research*, vol. 294, no. 2, pp. 405–426, 2021.

[2] International Organization for Standardization, "ISO 3691-4:2023. Industrial trucks – Safety requirements and verification — Part 4: Driverless industrial trucks and their systems," Standard, Geneva, Switzerland, 2023. [Online]. Available: https://www.iso.org/standard/83545.html

[3] R. B. Rusu and S. Cousins, "3D is here: Point cloud library (PCL)," in *2011 IEEE International Conference on Robotics and Automation*, 2011, pp. 1–4.

[4] G. Welch and G. Bishop, "An introduction to the Kalman filter," in *28th Annual Conference on Computer Graphics and Interactive Techniques*, 2001, course notes.

[5] J. Borenstein and Y. Koren, "The vector field histogram-fast obstacle avoidance for mobile robots," *IEEE Transactions on Robotics and Automation*, vol. 7, no. 3, pp. 278–288, 1991.

[6] D. Fox, W. Burgard, and S. Thrun, "The dynamic window approach to collision avoidance," *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23–33, 1997.

[7] L. Chen, P. Wu, K. Chitta, B. Jaeger, A. Geiger, and H. Li, "End-to-end autonomous driving: Challenges and frontiers," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 12, pp. 10 164–10 183, 2024.

[8] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in *2012 IEEE Conference on Computer Vision and Pattern Recognition*, 2012, pp. 3354–3361.

[9] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom, "nuScenes: A multimodal dataset for autonomous driving," in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 11 618–11 628.

[10] G. Papoudakis, F. Christianos, L. Schäfer, and S. Albrecht, "Benchmarking multi-agent deep reinforcement learning algorithms in cooperative tasks," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, vol. 1, 2021.

[11] D. Perille, A. Truong, X. Xiao, and P. Stone, "Benchmarking metric ground navigation," in *2020 IEEE International Symposium on Safety, Security, and Rescue Robotics*, 2020, pp. 116–121.

[12] J. Perez-Cerrolaza, J. Abella, M. Borg, C. Donzella, J. Cerquides, F. J. Cazorla, C. Englund, M. Tauber, G. Nikolakopoulos, and J. L. Flores, "Artificial intelligence for safety-critical systems in industrial and transportation domains: A survey," *ACM Computing Surveys*, vol. 56, no. 7, 2024.

[13] S. Macenski, T. Moore, D. V. Lu, A. Merzlyakov, and M. Ferguson, "From the desks of ROS maintainers: A survey of modern & capable mobile robotics algorithms in the robot operating system 2," *Robotics and Autonomous Systems*, vol. 168, p. 104493, 2023.

[14] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*. AAAI Press, 1996, p. 226–231.

[15] K. Katona, H. A. Neamah, and P. Korondi, "Obstacle avoidance and path planning methods for autonomous navigation of mobile robot," *Sensors*, vol. 24, no. 11, 2024.

[16] O. Pavliuk and M. Mishchuk, "Energy-aware spatio-temporal multi-agent route planning for agvs," *Applied Sciences*, vol. 16, no. 6, 2026.

[17] Y. Zhu, W. Z. Wan Hasan, H. R. Harun Ramli, N. M. H. Norsahperi, M. S. Mohd Kassim, and Y. Yao, "Deep reinforcement learning of mobile robot navigation in dynamic environment: A review," *Sensors*, vol. 25, no. 11, 2025.

[18] R. Bostelman, T. Hong, and G. Cheok, "Navigation performance evaluation for automatic guided vehicles," in *2015 IEEE International Conference on Technologies for Practical Robot Applications*, 2015, pp. 1–6.

[19] N. Shaker, J. Togelius, and M. J. Nelson, *Procedural content generation in games*, ser. Computational Synthesis and Creative Systems. Springer Cham, 2016.

[20] S. M. LaValle, *Planning Algorithms*. Cambridge University Press, 2006.